

```
pythonstyle language=Python, basicstyle=, keywordstyle=blue, commentstyle=gray,  
stringstyle=red, breaklines=true, frame=single, numbers=left, numberstyle=gray, backgroundColor=lightgray!20,
```

Complexity Analysis: Hadamard Test vs. Direct Expectation Value Calculation

1 Direct Expectation Value Calculation

Complexity: $O(N^3)$ in classical simulations.

Steps:

1. Prepare $|\psi_j\rangle = f_j|\phi_0\rangle$.
2. Apply H .
3. Compute $\langle\phi_0|w_i H f_j|\phi_0\rangle$ by evolving the state and extracting the probability amplitude.

Operations per matrix element: $O(N^2)$ single-qubit operations, followed by a classical probability computation.

2 Hadamard Test Complexity

Complexity: $O(N^4)$ for classical simulations due to repeated controlled operations and measurement statistics.

Steps:

1. Introduce an **ancilla qubit**.
2. Apply controlled-unitary gates for f_j , H , and w_i .
3. Use a **Hadamard gate** to extract the expectation value from the ancilla measurement.

Operations per matrix element:

- **Additional ancilla qubit overhead** ($O(1)$).
- **Controlled unitaries** increase the gate depth: $O(N)$ per term.
- **Repetitions** are needed for accurate measurement ($O(1/\epsilon^2)$ where ϵ is precision).
- **Measurement Sampling:** Classical post-processing requires **multiple shots** to estimate probabilities accurately.

3 Comparison of Resource Demand

Method	Circuit Depth	Number of Qubits	Classical Post-Processing
Direct Expectation	$O(N^2)$	$O(N)$	$O(N^3)$
Hadamard Test	$O(N^3)$	$O(N + 1)$	$O(N^4)$

Table 1: Comparison of Direct Expectation and Hadamard Test Complexity

4 Trade-offs

- **Direct Expectation is more efficient for classical simulations**, but does not generalize well to quantum hardware due to needing state vector inner products.
- **Hadamard Test is more demanding but hardware-compatible**, allowing real quantum devices to compute expectation values.

5 Conclusion

- **For classical simulation:** Use the **Direct Expectation strategy** for efficiency.
- **For quantum hardware:** Use the **Hadamard Test** to leverage quantum measurement.

6 Methodology

In this work, we employ a quantum computational approach to compute the **overlap matrix** and **Hamiltonian matrix elements** for non-orthogonal determinants in the context of quantum chemistry. The methodology follows a **statevector simulation framework** within Qiskit, leveraging Pauli string representations of fermionic operators to construct quantum circuits that evaluate the necessary matrix elements.

6.1 Representation of Determinants and Operators

A set of non-orthogonal determinants $\{|\psi_I\rangle\}$ is expressed in terms of fermionic creation and annihilation operators acting on a reference vacuum state $|\phi_0\rangle = |00000000\rangle$. These determinants are generated using the following relation:

$$|\psi_I\rangle = f_I|\phi_0\rangle, \quad \langle\psi_J| = \langle\phi_0|w_J \quad (1)$$

where f_I and w_J are sequences of fermionic operators mapped onto **Pauli strings** using the **Jordan-Wigner transformation**. The overlap matrix is then given by:

$$O_{IJ} = \langle\psi_I|\psi_J\rangle = \langle\phi_0|w_I f_J|\phi_0\rangle \quad (2)$$

Similarly, the Hamiltonian matrix elements are computed as:

$$H_{IJ} = \langle\psi_I|H|\psi_J\rangle = \langle\phi_0|w_I H f_J|\phi_0\rangle \quad (3)$$

where H represents the system’s Hamiltonian, decomposed into **one-body and two-body terms**. These are also encoded as weighted sums of **Pauli string operators**.

6.2 Quantum Circuit Construction and Evaluation

For each determinant pair (I, J) , we construct a quantum circuit that simulates the required transformation and measures the relevant expectation values. The **statevector approach** is utilized to ensure an exact simulation of the quantum evolution.

6.2.1 Loading the Operators

- The Pauli string representations of f_I , w_J , and H are loaded from precomputed files.
- Each file consists of a list of Pauli strings with corresponding coefficients.

6.2.2 Quantum Circuit Preparation

1. The system is initialized in the vacuum state $|\phi_0\rangle = |00000000\rangle$.
2. The determinant operator f_J is applied to create $|\psi_J\rangle$.
3. The Hamiltonian operator H (or identity in the case of the overlap matrix) is then applied.
4. Finally, the adjoint operator $w_I^\dagger$ is applied to obtain $|\psi_I\rangle$.

6.2.3 Expectation Value Computation

The expectation value of the **observable** is computed as:

$$\langle\psi_I|H|\psi_J\rangle = \text{Tr}(H \cdot |\psi_J\rangle\langle\psi_I|) \quad (4)$$

This is efficiently evaluated in Qiskit using `Statevector.expectation_value()`, which directly computes the expected outcome of measuring the applied operators in the computational basis.

6.3 Implementation Details

The implementation is carried out in Python using **Qiskit** and follows a structured pipeline:

- **Reading Pauli string files:** The function `read_pauli_file()` extracts the operator coefficients and Pauli representations from text files.
- **Constructing the Circuit:** The function `apply_pauli_string()` ensures that the correct sequence of Pauli operators is applied on the quantum circuit.
- **Computing Overlap and Hamiltonian Matrix Elements:**
 - For the **overlap matrix**, we compute the probability of measuring $|00000000\rangle$ after applying $w_I f_J$.
 - For the **Hamiltonian matrix**, we use `expectation_value()` to correctly obtain $\langle\psi_I|H|\psi_J\rangle$.
- **Saving the Results:** The computed matrices are stored as tab-separated text files for further analysis.

6.4 Conclusion

This methodology successfully computes both the **overlap matrix** and **Hamiltonian matrix elements** using quantum circuit simulation. The use of **Pauli string representations, expectation value computations, and statevector simulations** ensures an accurate and efficient implementation suitable for many-body quantum systems.

7 Why Was the Hadamard Test Not Used?

In quantum computing, the Hadamard test is a well-known technique used to compute expectation values of observables in a hardware-efficient manner. This method is particularly useful when working with real quantum hardware, where measuring expectation values directly is not straightforward. The Hadamard test utilizes an auxiliary qubit to estimate the real and imaginary parts of expectation values of an observable H for a given quantum state $|\psi\rangle$. The process involves applying a Hadamard gate to the ancilla qubit, followed by a controlled application of the unitary operator corresponding to the Hamiltonian evolution, and then measuring the ancilla in the computational basis.

Despite its effectiveness on quantum hardware, the Hadamard test was not implemented in this study because all calculations were performed using a statevector simulation. Unlike real quantum devices, where measurements are probabilistic, statevector simulations provide direct access to the full quantum state. This allows expectation values such as $\langle\psi_I|H|\psi_J\rangle$ to be computed exactly without requiring an additional ancilla qubit or controlled operations. In our implementation, this was achieved using Qiskit’s `Statevector` class, where the quantum circuit was evolved and the expectation value was computed directly with the function `expectation_value()`.

The choice of a statevector simulation over the Hadamard test was primarily motivated by efficiency and accuracy. The Hadamard test introduces additional circuit depth due to the need for controlled operations, which increases the complexity of quantum computations. Moreover, when executed on real quantum hardware, it requires statistical post-processing to extract expectation values, as quantum measurements are inherently noisy. By contrast, the statevector simulation used in this study provides exact results without the need for statistical averaging over multiple measurements. This approach is computationally more efficient for small systems where classical simulation remains feasible.

If the same calculations were to be executed on a real quantum device, the Hadamard test would be necessary because quantum hardware does not allow direct access to wavefunction amplitudes. Instead, expectation values would need to be estimated based on measurement statistics, and the Hadamard test would be the preferred method to achieve this. However, since all calculations in this study were performed in a classical quantum simulator, the Hadamard test was unnecessary, and the statevector method was chosen for its superior efficiency and accuracy.

Method	Why Use It?	Why Not Use It?
Hadamard Test	Needed for real quantum hardware where expectation values cannot be computed directly	Adds circuit depth, requires ancilla qubits, introduces measurement errors
Statevector Simulation	Exact computation of expectation values in classical simulation	Cannot be used on real quantum devices

Table 2: Comparison between the Hadamard test and statevector simulation

Since this study does not involve execution on a quantum processor, the statevector method was the optimal choice, providing accurate and computationally efficient results without the overhead associated with the Hadamard test.

8 Modifications for Real Quantum Hardware

The previous implementation used a **statevector simulation** to compute expectation values of the Hamiltonian matrix elements. While this approach is computationally efficient for small systems and provides exact results, it is not feasible for execution on real quantum hardware. Quantum computers do not allow direct access to wavefunction amplitudes, and all expectation values must be obtained through measurement-based methods. Therefore, we modify the implementation to use the **Hadamard test**, which is a widely used technique to estimate expectation values on a quantum processor.

The Hadamard test introduces an ancilla qubit that undergoes a Hadamard transformation before and after a controlled application of the unitary operator corresponding to the Hamiltonian. The expectation value of the Hamiltonian is then extracted from the probability distribution of measurement results on the ancilla qubit. By executing the quantum circuit on a real IBMQ backend and averaging over multiple shots, we obtain an estimate of the Hamiltonian matrix elements.

8.1 Comparison Between Statevector Simulation and Quantum Hardware Execution

8.2 Implementation Details

To adapt the code for quantum execution, we perform the following modifications:

Feature	Statevector Simulation	Real Quantum Hardware Execution
Expectation Value Computation	Direct evaluation using statevector math	Estimated via Hadamard test and measurement statistics
Quantum Backend	Qiskit <code>StatevectorSimulator</code> (classical)	IBM Quantum hardware using <code>IBMQ.load_account()</code>
Measurement Process	None (exact results)	Requires statistical averaging over multiple shots (e.g., 8192)
Circuit Depth	Shallow (fewer operations)	Deeper (includes controlled operations and Hadamard gates)
Accuracy	Exact and deterministic	Noisy due to quantum decoherence and gate errors

Table 3: Comparison between statevector simulation and real quantum hardware execution

1. Replace `Statevector.expectation_value()` with the Hadamard test.
2. Introduce an ancilla qubit for measurement-based computation.
3. Apply controlled operations in the Hadamard test circuit.
4. Execute the quantum circuit on the least busy `IBMQ` backend.
5. Estimate expectation values from measured results and compute the Hamiltonian matrix elements.

8.3 Modified Python Implementation

The following Python implementation includes these adaptations, ensuring that the computation can be executed on a real quantum device.

This implementation ensures that the Hamiltonian matrix elements are computed using quantum measurements instead of direct statevector calculations. The results obtained from `IBMQ` are then post-processed to estimate expectation values from measured statistics. By leveraging the Hadamard test and `IBMQ`'s quantum hardware, this approach enables the study of non-orthogonal determinant overlaps and Hamiltonian interactions on real quantum devices.